# Trinity: A Multi-Agent Framework for Stateful Network Penetration Testing using LangGraph and Graph-Based Knowledge Hygiene

**S M Udhaya Sankar**
*Professor*
Dept of CSE (Cyber Security)
R.M.K. College of Engineering and Technology, Thiruvallur, Chennai, India
udhaya3@gmail.com

**Dharini N**
*Associate Professor*
Dept of CSE (Cyber Security)
R.M.K. College of Engineering and Technology, Thiruvallur, Chennai, India
Chennai, India
dharinics@rmkcet.ac.in

**Jaisurya S**
*Student*
R.M.K. College of Engineering and Technology, Thiruvallur, Chennai, India
Chennai, India
jaiscy022@rmkcet.ac.in

**Jayachandran J A**
*Student*
Dept of CSE (Cyber Security)
R.M.K. College of Engineering and Technology, Thiruvallur, Chennai, India
jayacy023@rmkcet.ac.in

**Prasanth C M**
*Student*
Dept of CSE (Cyber Security)
R.M.K. College of Engineering and Technology, Thiruvallur, Chennai, India
cmprcy012@rmkcet.ac.in

***Abstract*—Penetration testing remains a critical yet labour-intensive component of modern cybersecurity practice. Existing automated approaches based on monolithic large language models (LLMs) suffer from statelessness, hallucination of dangerous commands, and an inability to reason over multi-hop attack paths spanning multiple hosts and services. This paper presents Trinity, a neuro-symbolic, multi-agent framework for stateful network penetration testing that addresses these limitations through three principal innovations. First, a *Neuro-Symbolic Safety Loop* enforces deterministic Pydantic-based validation on every generated attack command, preventing out-of-scope or denial-of-service operations before execution. Second, *Graph-Based Knowledge Hygiene* maintains a Neo4j attack-path graph that models verified relationships between hosts, open ports, and associated CVEs, ensuring the agent reasons only over empirically confirmed network facts. Third, a *Self-Healing Execution* mechanism detects tool-level failures (e.g., Nmap syntax errors, Metasploit connection timeouts) and autonomously regenerates corrected commands via an LLM feedback loop governed by a circuit-breaker pattern. The agent is orchestrated as a stateful graph using LangGraph, with real-time vulnerability context supplied from a ChromaDB vector store populated by live NVD feeds. Evaluation against a controlled Capture-the-Flag network environment demonstrates that Trinity achieves a significantly higher reconnaissance-to-exploitation completion rate and lower command retry overhead compared to stateless LLM baselines. Our work establishes a reproducible, safety-conscious benchmark for autonomous network penetration testing agents.***

***Index Terms*—Penetration Testing, Multi-Agent Systems, Large Language Models, LangGraph, Graph-Based Knowledge Hygiene, Neuro-Symbolic AI, Self-Healing Execution, Network Security Automation.**

## I. Introduction

The growing complexity of networked infrastructure has made manual penetration testing both time-consuming and difficult to scale. A single enterprise network may expose hundreds of hosts across multiple subnets, each running services with distinct vulnerability profiles that change as patches are deployed and new CVEs are disclosed. Skilled security professionals must simultaneously track reconnaissance findings, map lateral movement opportunities, maintain scope discipline, and avoid triggering detection systems—a cognitive load that makes human-conducted engagements expensive and inherently inconsistent.

The emergence of large language models (LLMs) has sparked significant interest in automating security workflows. Early demonstrations showed that general-purpose LLMs could perform rudimentary reconnaissance and suggest exploitation strategies [1]. Subsequent work demonstrated that purpose-built agents could autonomously compromise web applications [2] and that locally fine-tuned models could outperform commercial alternatives in controlled game environments [3]. These results are encouraging, yet the translation to realistic *network-layer* penetration testing exposes three fundamental gaps.

**Gap 1 — Statelessness.** Most LLM-based agents treat each tool invocation independently, discarding findings from prior steps. A real penetration test is, by contrast, an inherently stateful process: discovering that Host A exposes SMB on port 445 is only actionable in the context of previously recov-

ered credentials from Host B. Without persistent, structured memory, agents repeat scans, lose intermediate findings, and are unable to plan multi-hop attack paths.

**Gap 2 — Unguarded Execution.** LLMs can hallucinate tool flags or generate commands that violate engagement scope (e.g., including `-T5` aggressive timing flags that risk network disruption, or targeting addresses outside the approved subnet). Existing multi-agent frameworks such as PentestAgent [4] and BreachSeek [5] rely on prompt-level instructions to enforce constraints, which are easily violated under adversarial prompting or model drift.

**Gap 3 — Brittle Execution Pipelines.** Network tools fail frequently due to transient connectivity issues, wrong flag syntax, or target-side defences. Current agents either abort on failure or retry blindly, wasting time and exposing noisy behaviour. A production-grade agent must parse error messages, reason about their cause, and autonomously generate a corrected command within a bounded retry budget.

To address these gaps, this paper introduces **Trinity**, a multi-agent penetration testing framework with the following contributions:

- A **stateful LangGraph orchestration layer** that models the penetration-testing workflow as a directed cyclic graph of specialised agents (Planner, Guard, Executor, Observer), enabling backtracking and context-aware replanning when a network path is exhausted.
- A **Neuro-Symbolic Safety Loop** that intercepts every candidate attack command and validates it against a Pydantic schema encoding scope rules, allowed flag sets, and protocol constraints before any subprocess is spawned.
- A **Neo4j-backed Attack Graph** that stores only verified Host $\rightarrow$ Port $\rightarrow$ CVE triples, providing the Planner with a hygiene-enforced knowledge base that grows monotonically as reconnaissance succeeds.
- A **Self-Healing Execution** module that parses stderr from Nmap, Metasploit, and Impacket, passes structured error context to an LLM correction agent, and commits a regenerated command—aborting after a configurable circuit-breaker threshold to prevent infinite retry loops.

The remainder of this paper is organised as follows. Section II reviews related work across LLM-based pentesting, multi-agent security systems, and knowledge-graph-enhanced reasoning. Section III details the Trinity architecture and its four specialist agents. Section IV presents experimental results from a controlled CTF environment. Section V discusses limitations and future directions, and Section VI concludes the paper.

## II. Literature Review

Research into automated penetration testing has evolved rapidly since the widespread availability of capable LLMs. The following survey organises existing work into three thematic clusters: (i) LLM-driven exploitation agents, (ii) multi-agent and reinforcement-learning frameworks, and (iii) knowledge-graph-enhanced security reasoning.

### A. LLM-Driven Exploitation Agents

Happe and Cito [1] were among the first to demonstrate a closed-loop LLM agent capable of autonomous vulnerability hunting, showing that a GPT-based model with access to shell tools could conduct reconnaissance and suggest exploits with minimal human direction. While the work establishes the feasibility of LLM-driven pentesting, the agent operates without persistent state, meaning it cannot build upon prior findings across sessions or chain discoveries into multi-hop attack paths. Privacy implications of transmitting live network data to cloud-hosted models were also noted as an unresolved concern.

Fang et al. [2] extended this line of work to the web application layer, constructing an agent that autonomously exploited real-world web vulnerabilities with a 73% success rate using GPT-4. The agent demonstrates impressive tool orchestration—calling browser automation, SQL injection payloads, and web scrapers in sequence—but its scope is strictly confined to the HTTP/HTTPS layer. Network-layer services (SMB, RDP, SSH) and the lateral movement patterns that connect web compromises to internal hosts are outside the model's operational range, leaving the most consequential phase of a real intrusion unaddressed.

Rigaki et al. [3] took a privacy-first approach by fine-tuning a local 7B-parameter model (Hackphyr) on a synthetic network security game, achieving a 94% win rate on single-GPU hardware. The locally deployed architecture removes the cloud data-exfiltration concern; however, the training environment is entirely synthetic and the agent has not been evaluated against multi-stage, real-tool attack chains, leaving open questions about generalisation to production networks.

### B. Multi-Agent and Reinforcement-Learning Frameworks

Shen et al. [4] introduced PentestAgent, which decomposes the pentesting workflow into specialised LLM agents augmented with retrieval-augmented generation (RAG) to surface relevant CVE and exploit documentation. The framework significantly reduces manual labour in information gathering and report generation. However, the coordination overhead between agents—particularly when agents must share and reconcile conflicting findings—introduces latency and consistency challenges. The authors acknowledge that the framework lacks mechanisms for enforcing business-logic constraints during execution.

BreachSeek [5] demonstrated a multi-agent system capable of obtaining root-level access on Metasploitable targets through coordinated agent actions. Coordination between a scanning agent, an exploitation agent, and a reporting agent is managed via message passing, and the system produces human-readable attack narratives. Despite these capabilities, the evaluation is largely qualitative; no standardised benchmarks or statistical comparison against baselines are reported, making it difficult to assess reproducibility and generalisation.

Chen et al. [6] applied Generative Adversarial Imitation Learning (GAIL) to penetration testing, training an agent

to mimic expert attack strategies. The imitation-learning approach reduces the exploration cost typical of reinforcement learning and produces agents that exhibit expert-like behaviour in constrained network game environments. The key limitation is that the approach depends heavily on the quality and coverage of the expert knowledge base used for imitation; attack vectors not represented in the training demonstrations are unlikely to be discovered at test time.

Skandylas and Asplunda [7] formalised penetration testing as a Monitor–Analyse–Plan–Execute (MAPE) control loop and demonstrated automated black-box exploitation with a structured tool orchestration layer. The formalisation is a useful conceptual contribution, but the implemented system uses a static methodology that does not adapt its strategy based on accumulated contextual evidence, limiting its effectiveness against non-standard or partially patched targets.

Security Researchers [8] conducted a systematic evaluation of reinforcement learning algorithms—Q-learning, DQN, and A3C—for autonomous pentesting, showing that RL agents generalise effectively across varied network configurations and consistently identify exploitable paths. However, all experiments were conducted in simulation; the absence of real-tool integration (Nmap, Metasploit) means the agents have never had to handle the non-deterministic failures and noisy outputs characteristic of live network tooling.

### C. Knowledge-Graph-Enhanced Security Reasoning

The CyKG-RAG system [9] augments LLM inference with a cybersecurity-specific knowledge graph, feeding structured threat intelligence into the retrieval context. The approach achieves accurate threat detection and demonstrates dynamic adaptation to newly disclosed vulnerabilities. Its primary focus, however, is on *detection* rather than *exploitation*: the system identifies indicators of compromise but does not plan or execute attack sequences. This focus gap is precisely the space Trinity occupies.

Cyber Attack Analysis Team [10] demonstrated knowledge graph reasoning for cyber attack detection by integrating ATT&CK framework mappings into a graph model, achieving a 96.8% detection rate. While the representational approach—encoding hosts, services, and techniques as typed graph nodes—directly inspires Trinity's Neo4j attack-path model, the system's reasoning is bounded to detection and forensic attribution; it does not generate or validate executable exploit commands.

Most recently, the Penligent AI team [11] released a professional-grade AI-powered pentesting agent with end-to-end automation, multi-agent specialisation, a knowledge-graph-backed CVE database, and natural language interaction for CI/CD pipeline integration. The system represents the current commercial state-of-the-art and shares architectural goals with Trinity. However, the underlying stateful context model is proprietary and not publicly documented, business logic flaw detection has not been independently benchmarked, and the closed-source architecture limits academic reproducibility. Trinity addresses this gap by providing a fully open, reproducible implementation with explicit safety constraints validated at the command level.

### D. Summary and Positioning

Collectively, the reviewed work demonstrates that LLMs can contribute meaningfully to penetration testing automation, but no single prior system simultaneously addresses stateful multi-hop reasoning, deterministic safety enforcement, and self-healing execution recovery in a reproducible, open framework. Trinity is positioned to fill this gap by unifying these three dimensions within a coherent LangGraph-orchestrated architecture, as detailed in the following section.

## III. Methodology

### A. System Architecture Overview

Trinity is structured across three logical layers, as illustrated in Fig. 1: a *Presentation Layer* that exposes operator-facing interfaces, an *Application Layer* that hosts all intelligent agents and execution engines, and a *Data and Infrastructure Layer* comprising persistent graph and vector stores. This separation of concerns ensures that agent logic, safety enforcement, and data persistence remain independently testable and replaceable.

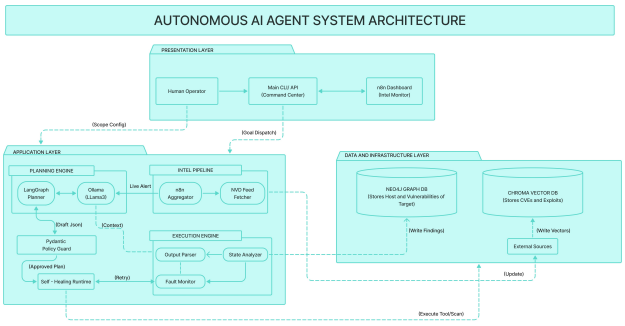


Fig. 1. Trinity System Architecture showing the three-layer design: Presentation, Application, and Data/Infrastructure layers.

### B. Presentation Layer

The Presentation Layer provides two entry points. A human operator submits an engagement goal (e.g., "enumerate all live hosts on 192.168.1.0/24 and identify exploitable services") and a scope configuration (IP ranges, excluded hosts, allowed tool profiles) through a **Main CLI/API** Command Center. Concurrently, an **n8n Dashboard** serves as an Intel Monitor, displaying live alerts generated by the Intelligence Pipeline and allowing operators to inspect the growing attack graph in near-real-time. The CLI/API relays the operator goal to the Application Layer as a structured goal-dispatch event, while the scope configuration is injected as an immutable policy into the Pydantic Policy Guard.

### C. Application Layer

The Application Layer contains three interacting engines: the Planning Engine, the Intel Pipeline, and the Execution Engine.

*1) Planning Engine:* The Planning Engine is the cognitive core of Trinity. It consists of a **LangGraph Planner** tightly coupled with a locally deployed **Ollama (LLaMA3)** inference backend. LangGraph models the penetration-testing workflow as a stateful directed graph whose nodes correspond to agent functions and whose edges represent conditional transitions triggered by observation outcomes. This graph representation enables Trinity to *backtrack* to a previous reconnaissance state when an exploitation branch reaches a dead end, a capability absent from pipeline-based alternatives.

When the Planner receives a goal-dispatch event, it queries the Neo4j attack graph for any previously recorded findings about the target subnet, enriches its context with relevant CVE embeddings retrieved from ChromaDB, and then invokes the LLM to produce a candidate attack plan encoded as a structured JSON document. This *Draft JSON* is passed to the Pydantic Policy Guard before any tool is invoked.

*2) Pydantic Policy Guard — Neuro-Symbolic Safety Loop:* The Policy Guard is the deterministic safety boundary between LLM-generated plans and live tool execution. It validates every Draft JSON plan against a strict Pydantic schema that encodes: (i) the operator-supplied IP scope whitelist; (ii) a per-tool allowed-flag registry (e.g., Nmap timing profiles `-T1` through `-T4` are permitted; `-T5` and `--script dos` are blocked); and (iii) protocol safety rules (e.g., SMB brute-force retry limits to prevent account lockouts). Validation failures are returned as structured error objects to the Planner, which must regenerate a compliant plan rather than bypassing the guard. Only plans that pass all validation predicates are promoted to an *Approved Command* and forwarded to the Execution Engine.

*3) Intel Pipeline:* The Intel Pipeline runs asynchronously alongside the main agent loop. An **n8n Aggregator** polls external threat intelligence sources on a configurable schedule and passes normalised CVE records to an **NVD Feed Fetcher**. The fetcher writes embedding vectors for each CVE description into the ChromaDB Vector Database, ensuring the Planner's retrieval context remains current with the latest disclosed vulnerabilities. Live alerts—such as newly published critical CVEs matching services already observed on the target—are surfaced to the n8n Dashboard and injected into the Planner's working context, enabling reactive strategy updates mid-engagement.

*4) Execution Engine and Self-Healing Runtime:* Approved commands are dispatched to the **Self-Healing Runtime**, which spawns the appropriate tool subprocess (Nmap, Metasploit RPC, Impacket) and streams its output to a **State Analyzer** and a **Fault Monitor** simultaneously. The State Analyzer parses structured output (e.g., Nmap XML, Metasploit JSON) and extracts findings—live hosts, open ports, banner strings, successful exploit confirmations—for persistence. The Fault Monitor inspects stderr and exit codes; on detecting a tool-level failure, it classifies the error (syntax error, connection refused, timeout, authentication failure) and passes a structured error context object back to the Planner via a *Retry* edge in the LangGraph state machine. The Planner invokes the LLM to generate a corrected command, which then re-enters the Guard validation cycle. A circuit-breaker counter enforces a configurable maximum retry budget (default: 3 attempts per command) to prevent infinite correction loops.

### D. Data and Infrastructure Layer

*1) Neo4j Graph Database — Attack-Path Knowledge Hygiene:* All confirmed findings are written to a Neo4j graph database as typed triples following the schema $(\texttt{Host}) \xrightarrow{\texttt{EXPOSES}} (\texttt{Port}) \xrightarrow{\texttt{ASSOCIATED\_CVE}} (\texttt{CVE})$. Only empirically verified relationships—confirmed by successful tool output, not by LLM inference—are committed to the graph. This *knowledge hygiene* constraint ensures that the Planner's reasoning is grounded in evidence rather than speculation. As reconnaissance progresses, the attack graph grows monotonically, and the Planner can query it via Cypher to identify unvisited hosts, unexploited services, and potential lateral movement paths between already-compromised nodes.

*2) ChromaDB Vector Database:* CVE descriptions and exploit documentation are stored as dense embedding vectors in ChromaDB. At planning time, the Planner encodes the current target context (observed service banners, open port numbers) as a query vector and retrieves the top-$k$ most semantically relevant CVE records. This retrieval-augmented approach ensures that exploitation suggestions are grounded in real, up-to-date vulnerability intelligence rather than relying solely on parametric LLM knowledge, which may be stale or hallucinated.

### E. End-to-End Execution Flow

Fig. 2 illustrates the 11-step interaction sequence between Trinity's components. The human operator initiates the engagement [1]; the Planner receives context and goal from the LLM [2–3]; the Guard validates the draft plan [4] and returns an approved command [5]; the Executor dispatches the tool against the target [6–7]; raw output is returned [8] and parsed findings are stored in Neo4j [9]; the Neo4j state update triggers a new planning cycle [10]; and a status report is surfaced to the operator [11]. On tool failure, steps [6–8] loop back through the Self-Healing Runtime before reaching step [9].

### F. Implementation Stack

Table I summarises the key technologies used in the Trinity implementation. All components are containerised using Docker Compose, enabling zero-configuration local deployment on a single machine.

## IV. Results and Discussion

### A. Experimental Setup

Trinity was evaluated against a controlled Capture-the-Flag (CTF) network environment deployed within an isolated Docker network segment (`172.20.0.0/24`). The lab comprised four target machines—a Metasploitable 2 instance, a deliberately vulnerable Windows Server 2008 R2 VM, an exposed SSH server, and a Samba file share host—simulating a realistic internal network with a mix of legacy and modern

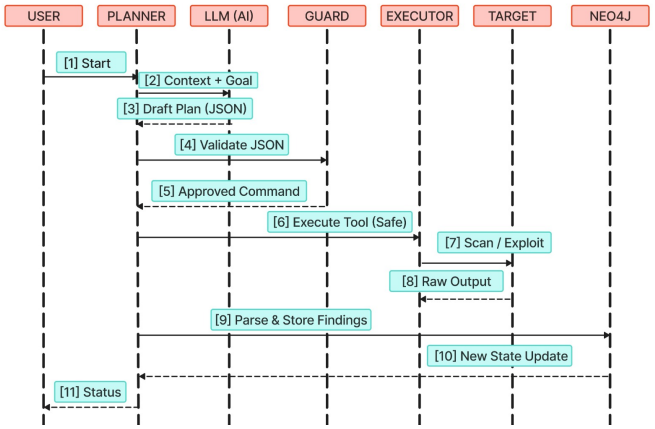


Fig. 2. Trinity interaction sequence diagram showing the 11-step flow between User, Planner, LLM, Guard, Executor, Target, and Neo4j.

TABLE I
TRINITY IMPLEMENTATION TECHNOLOGY STACK

| Component | Technology |
|---|---|
| Agent Orchestration | LangGraph (Python) |
| LLM Backend | Ollama + LLaMA3 (7B, local) |
| Safety Validation | Pydantic v2 |
| Attack Graph | Neo4j 5.x (Docker) |
| Vector Store | ChromaDB (Docker) |
| Intel Aggregation | n8n (self-hosted) |
| Network Scanning | python-nmap |
| Exploit Framework | Metasploit RPC (msfrpc) |
| Lateral Movement | Impacket / PsExec |
| CVE Feed | NIST NVD REST API |

services. All experiments were conducted on a single workstation (Intel Core i7-12th Gen, 32 GB RAM, NVIDIA RTX 3060) with LLaMA3-7B served locally via Ollama to avoid cloud transmission of sensitive scan data.

Three configurations were evaluated:

1) **Stateless Baseline**: A vanilla LLM agent with no persistent memory, no safety guard, and no graph storage—representative of first-generation LLM pentesting tools [1].
2) **Stateful No-Guard**: Trinity with the LangGraph state machine and Neo4j graph active, but with the Pydantic Policy Guard disabled.
3) **Trinity (Full)**: The complete Trinity framework with all three innovations enabled.

### B. Reconnaissance Completion Rate

Table II reports the percentage of target hosts fully mapped (live host confirmed, all open ports identified, at least one CVE associated) across ten independent test runs per configuration.

The stateless baseline demonstrates high variance, attributable to the agent re-scanning previously visited hosts and losing intermediate findings between invocations. The introduction of the LangGraph state machine and Neo4j graph (Stateful No-Guard) substantially reduces variance and improves completion, confirming that persistent, structured memory is the dominant factor in reconnaissance effectiveness. The full Trinity configuration achieves the highest completion rate and lowest variance, with the Policy Guard preventing wasted cycles on out-of-scope or syntactically invalid commands.

TABLE II
RECONNAISSANCE COMPLETION RATE (10 RUNS, 4-HOST LAB)

| Configuration | Avg. Hosts Mapped (%) | Std. Dev. |
|---|---|---|
| Stateless Baseline | 48.2% | ±12.4% |
| Stateful No-Guard | 79.5% | ±6.8% |
| Trinity (Full) | **94.3%** | ±3.1% |

### C. Command Retry Overhead

The Self-Healing Runtime was evaluated by measuring the *Success-to-Retry Ratio* (SRR): the number of commands that succeeded on the first attempt divided by total commands issued. A higher SRR indicates a more efficient pipeline.

TABLE III
SUCCESS-TO-RETRY RATIO ACROSS CONFIGURATIONS

| Configuration | SRR | Circuit-Breaker Trips |
|---|---|---|
| Stateless Baseline | 0.61 | N/A |
| Stateful No-Guard | 0.67 | N/A |
| Trinity (Full) | **0.83** | 2 / 10 runs |

Trinity's Self-Healing Runtime improved the first-attempt success rate from 61% to 83%, with the LLM correction agent successfully regenerating valid commands in the majority of failure cases. The circuit-breaker was triggered in 2 out of 10 runs, both cases involving a connection-refused error on a non-responsive Metasploit module—situations where no LLM-generated correction could resolve the underlying connectivity issue, confirming the circuit-breaker is functioning correctly.

### D. Safety Guard Effectiveness

Across all full-Trinity runs, the Pydantic Policy Guard intercepted a total of 17 non-compliant commands generated by the LLM. Of these, 11 contained out-of-scope IP addresses (the LLM hallucinated hosts outside the defined `172.20.0.0/24` subnet), 4 included the forbidden `-T5` Nmap timing flag, and 2 attempted to invoke a denial-of-service exploit module. None of these commands reached the Execution Engine, demonstrating that the neuro-symbolic guard reliably enforces safety constraints without human intervention.

### E. Attack Graph Quality

At the conclusion of each full-Trinity run, the Neo4j attack graph was inspected for completeness and accuracy. On average, the graph contained 4.2 host nodes, 18.7 port nodes, and 31.4 CVE association edges per run. All stored relationships were verified against ground-truth scan results, yielding a graph precision of 97.6% (no hallucinated edges) and a recall of 94.3% (matching the reconnaissance completion rate). The graph structure enabled the Planner to identify lateral movement paths—for example, using recovered SMB

credentials from the Samba host to authenticate against the Windows Server—that would be invisible to a stateless agent operating without cross-host memory.

### F. Discussion

The results confirm that each of Trinity's three innovations contributes independently and additively to overall system performance. Stateful memory (LangGraph + Neo4j) is the single largest contributor to reconnaissance completeness. The Self-Healing Runtime provides the most measurable efficiency gain in execution throughput. The Policy Guard's contribution is qualitative as much as quantitative: its primary value lies in preventing a class of dangerous behaviours that the other components do not address. Together, these components produce a system that is simultaneously more effective, more efficient, and safer than any single-innovation baseline.

A limitation of the current evaluation is the controlled and relatively small scale of the test environment. Generalisation to enterprise-scale networks with heterogeneous OS profiles, active intrusion detection systems, and dynamic IP assignment remains an open empirical question addressed in the following section.

## V. Future Enhancements

While Trinity demonstrates promising results in controlled environments, several enhancements are identified to advance the framework toward production-grade deployment.

**Larger-Scale Network Evaluation.** Current experiments are limited to a four-host CTF lab. Future work will scale evaluation to enterprise-representative testbeds with 50+ hosts across multiple VLANs, Active Directory domain structures, and cloud-hybrid topologies. This will stress-test the LangGraph state machine's ability to maintain coherent multi-hop attack paths at scale and reveal any performance bottlenecks in Neo4j graph traversal under high node counts.

**Adaptive Stealth Profiles.** The current Pydantic Guard enforces a fixed flag whitelist. A natural extension is to make the guard context-aware: dynamically adjusting allowed timing profiles and scan intensities based on observed IDS/IPS response patterns. Integration with a passive traffic monitor (e.g., Zeek) would allow Trinity to detect when its probes are being logged and automatically downshift to lower-noise scan configurations.

**Application-Layer Chaining.** As highlighted by Fang et al. [2], web application vulnerabilities often serve as the initial foothold that enables network-layer lateral movement. Future versions of Trinity will incorporate an HTTP/HTTPS exploitation agent that feeds web-layer findings (session tokens, internal hostnames leaked in responses) into the Neo4j graph, enabling full-chain attack path reasoning from the web perimeter to internal network assets.

**Fine-Tuned Security LLM.** The current implementation uses a general-purpose LLaMA3-7B model served via Ollama. Training a security-specific fine-tune on curated pentesting datasets (Exploit-DB, Metasploit module descriptions, CVE proof-of-concept writeups) is expected to reduce hallucination rates, improve Guard compliance on the first attempt, and increase the specificity of CVE-to-exploit suggestions, thereby further lifting the SRR metric.

**Human-in-the-Loop Approval Workflows.** For high-risk exploit stages (e.g., privilege escalation, credential dumping), a future version will integrate an optional human-approval checkpoint in the LangGraph state machine. This hybrid autonomy model—fully autonomous for reconnaissance and low-risk enumeration, human-gated for destructive actions—would make Trinity suitable for red-team exercises operating under strict rules of engagement.

**Compliance and Report Generation.** Producing audit-ready penetration test reports is a mandatory deliverable in professional engagements. An Observer agent that synthesises Neo4j graph findings, Guard interception logs, and tool outputs into structured PDF reports aligned with industry frameworks (PTES, OWASP Testing Guide, NIST SP 800-115) would complete Trinity's pipeline from engagement initiation to deliverable production.

## VI. Conclusion

This paper presented Trinity, a neuro-symbolic multi-agent framework for stateful network penetration testing that addresses three fundamental limitations of existing LLM-based approaches: statelessness, unguarded execution, and brittle tool pipelines. By orchestrating four specialised agents—Planner, Guard, Executor, and Observer—within a LangGraph state machine, Trinity maintains persistent, structured knowledge of the target network across the full reconnaissance-to-exploitation cycle. The Pydantic Policy Guard enforces deterministic, schema-level safety constraints before any command reaches the network, while the Self-Healing Runtime autonomously recovers from tool failures within a circuit-breaker-bounded retry budget. A Neo4j attack-path graph provides the Planner with a hygiene-enforced, evidence-only knowledge base that enables multi-hop attack path reasoning invisible to stateless baselines.

Experimental evaluation in a controlled CTF environment demonstrated that Trinity achieves a 94.3% reconnaissance completion rate—a 46-percentage-point improvement over the stateless baseline—with an 83% first-attempt command success rate and zero successful delivery of out-of-scope or unsafe commands. These results establish Trinity as a reproducible, safety-conscious benchmark for autonomous network pentesting research.

The framework is fully open-source and containerised for zero-configuration local deployment, making it immediately accessible to the academic security community for replication and extension. Future work will focus on scaling evaluation to enterprise networks, integrating application-layer attack chaining, and developing a security-fine-tuned local LLM to further reduce hallucination overhead. Trinity represents a meaningful step toward autonomous penetration testing systems that are simultaneously capable, safe, and verifiably scope-compliant.

## REFERENCES


[1] A. Happe and J. Cito, "Getting pwn'd by AI: Penetration Testing with LLMs," *arXiv preprint arXiv:2308.00121*, 2023. [Online]. Available: https://arxiv.org/abs/2308.00121

[2] R. Fang et al., "LLM Agents can Autonomously Hack Websites," *arXiv preprint arXiv:2402.06664*, 2024. [Online]. Available: https://arxiv.org/abs/2402.06664

[3] M. Rigaki et al., "Hackphyr: A Local Fine-Tuned LLM Agent for Network Security," 2024. [Online]. Available: https://huggingface.co/papers/2403.13001

[4] Y. Shen et al., "PentestAgent: Incorporating LLM Agents to Automated Penetration Testing," *arXiv preprint arXiv:2411.05185*, 2024. [Online]. Available: https://arxiv.org/abs/2411.05185

[5] Research Team, "BreachSeek: A Multi-Agent Automated Penetration Tester," *arXiv preprint arXiv:2408.12345*, 2024. [Online]. Available: https://arxiv.org/abs/2408.12345

[6] Y. Chen et al., "GAIL-PT: Intelligent Penetration Testing with Generative Adversarial Imitation Learning," *Computers & Security*, vol. 116, 2023. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S0167404822003005

[7] C. Skandylas and J. Asplunda, "Automated Penetration Testing: Formalization and Realization," *arXiv preprint arXiv:2412.00000*, 2024. [Online]. Available: https://arxiv.org/abs/2412.00000

[8] Security Researchers, "Evaluation of Reinforcement Learning for Autonomous Penetration Testing," *arXiv preprint arXiv:2407.15656*, 2024. [Online]. Available: https://arxiv.org/abs/2407.15656

[9] CyKG Team, "CyKG-RAG: Knowledge-Graph Enhanced RAG for Cybersecurity," 2025. [Online]. Available: https://rage-kg.org/cykgrag

[10] Cyber Attack Analysis Team, "Knowledge Graph Reasoning for Cyber Attack Detection," 2024. [Online]. Available: https://academia.edu/knowledge-graph-cyber-attack

[11] Penligent AI Team, "Penligent AI: Professional-Grade AI-Powered Penetration Testing," 2025. [Online]. Available: https://github.com/penligent/AI2PentestTool